# PES University

Department of Computer Science & Engineering

UE25CS642B - Distributed System: Batch 6

# AdaptaKV

## Adaptive Replicated Key-Value Store

— Toggle Between Availability & Consistency | Measure Latency vs Correctness Under Partition —

**Theme B: CAP Theorem | Problem B.2**


| Name | Roll Number |
|---|---|
| ABHAY G MOUDHGAL | PES1PG25CS001 |
| RAGULA UDAY | PES1PG25CS054 |
| SUNKU PEDA AKSHAY | PES1PG25CS067 |
| REVANTHESH C | PES1PG25CS112 |

Mentored By: Nitin V Pujari
Faculty, Computer Science | Dean – IQAC,
PES University

# 1. Problem Statement

Build a simple replicated key-value store where the user can toggle between availability or consistency priority. Measure latency vs. correctness under partition.

| # | Objective | Description |
|---|---|---|
| ① | Replicated KV Store | A multi-node key-value store with active replication across nodes. |
| ② | Toggle Mode | Runtime switch between CP mode (consistency priority) and AP mode (availability priority) with near to zero downtime. |
| ③ | Partition Simulation | Simulate network partitions (latency / packet loss) and observe divergence in state across replicas. |
| ④ | Measure Latency vs Correctness | Instrument both read/write latency and stale-read anomaly counts (Divergence Index) under partition. |

# 2. Real-Life Example: Flash Sale Inventory System

Transaction isolation and replication anomalies appear in every e-commerce, banking, and ticket-booking system under concurrent load. AdaptaKV addresses the common real-world need to balance speed and correctness during transient network failures.

| Phase | Mode | Concern | Anomaly Risk | AdaptaKV Behavior |
|---|---|---|---|---|
| Pre-sale | CP Mode | Inventory accuracy critical | LOW | W=2, R=2 quorum — no overselling. |
| Network Partition | Auto-detect | Two nodes split from one | HIGH without AdaptaKV | DI spikes → alert. Toggle to AP to continue serving. |
| Peak traffic | AP Mode | Latency < 50ms required | MEDIUM | W=1, R=1. Scorer tracks DI in real time. |
| Post-sale reconcile | CP Mode | Final counts must agree | LOW | Toggle back to CP. WAL replay repairs divergent replicas. |

# 3. Our Unique Solution: AdaptaKV

AdaptaKV is a single-binary replicated key-value store that dynamically rewrites its own quorum policy at runtime via a gRPC control plane — without restarting nodes. This is what makes it novel: existing production KV stores (etcd, Consul, Redis) commit to a single consistency model at deploy time. AdaptaKV makes the quorum policy a live, observable variable.

## 3.1 The Divergence Index (DI)

A background Correctness Scorer thread periodically reads the same key from all N replicas and computes:

**DI = (number of replicas whose value differs from primary) / N**

DI = 0 means full consistency. DI = 1 means every replica disagrees with the primary. DI is exposed as a Prometheus gauge and graphed live in Grafana alongside write_latency_p99, giving operators a numeric handle on the consistency-latency trade-off at every moment.

## 3.2 Live Quorum Toggle

The gRPC ControlService exposes SetMode(CP | AP). On receipt:

- The etcd key /adaptakv/mode is updated atomically.
- All 3 KV nodes watch this key — they receive the change within milliseconds.
- Each node recomputes: CP → W = ⌈N/2⌉+1, R = ⌈N/2⌉+1. AP → W = 1, R = 1.
- Mid-flight writes complete under the old policy; new writes use the new quorum.

## 3.3 Partition-Aware Replication

NATS JetStream carries replication events (kv.replicate subject) with HLC timestamps. When a node is partitioned and reconnects, it replays all missed JetStream events in HLC order — no manual reconciliation needed.

# 4. Proposed Solution

AdaptaKV integrates four core subsystems:

| # | Component | Description |
|---|---|---|
| 1 | Multi-Session KV Node (Go) | Each node is a Go binary with RocksDB backend. goroutines handle concurrent client requests. sync. RWLock per shared enables AP read scaling. |
| 2 | gRPC Control Plane | SetMode() RPC triggers etcd watch → all nodes recompute quorum. Mode change is atomic and near to zero-downtime. |
| 3 | NATS JetStream Replication Bus | Primary publishes every put/delete to kv.replicate. Replicas consume and apply. JetStream retains history for partition recovery. |
| 4 | Correctness Scorer + Grafana | Background goroutine polls all replicas for the same key every 500ms. Computes DI. Pushes stale_reads_total and divergence_index to Prometheus. Grafana renders latency vs DI. |

# 5. Technology Stack

Each of the 13 required layers has one chosen technology with concrete justification and comparative analysis of rejected alternatives.

| Layer / Concern | Chosen Technology | One-Line Rationale |
|---|---|---|
| Programming Language | Go | goroutines give true concurrent replica sessions; net/http + gRPC are first-class; sync.RWLock enables per-shard AP/CP control. |
| Communication / RPC | gRPC | Bidirectional streaming for live toggle signals; Protobuf-typed service definitions prevent stringly-typed errors. |
| Message Bus (Sagas) | NATS (JetStream) | Lightweight pub-sub with persistence; JetStream retains history so partitioned replicas replay missed events on reconnect. |
| Storage Engine (Local) | RocksDB | LSM-tree for high-throughput AP writes; native WAL provides before/after state; column families separate data and replication log. |
| Distributed Coordination | etcd | etcd watches drive atomic quorum reconfiguration; leases detect node failures for automatic DI recalculation. |
| Logging / WAL | slog (Go stdlib) | Zero-dependency structured JSON logger; async-safe; {key, before, after, hlc, mode} log is the WAL equivalent. |

| | | |
|---|---|---|
| Serialization | Protobuf | Schema-enforced binary encoding; ~5x smaller than JSON on replication hot path; schema evolution without breaking replicas. |
| Concurrency Control | Custom + sync.RWLock | 256-shard map with one RWLock per shard: readers never block each other in AP; writes serialized per-shard in CP. |
| Time / Timestamp Oracle | Hybrid Logical Clock (HLC) | HLC = max(wall, last_seen) + counter; monotonic across NTP jumps; used by Scorer to detect stale reads by HLC comparison. |
| Testing / Simulation | Toxiproxy | TCP proxy injects deterministic latency/drops between replica containers; REST-controlled from Go test code; reproducible. |
| Metrics / Dashboard | Prometheus + Grafana | Go client_golang counters: stale_reads_total, divergence_index, write_latency_seconds{mode}; Grafana renders live latency vs DI. |
| Container / Orchestration | Docker + Compose | docker-compose.yml defines all 8 services; docker compose up reproduces the full cluster on any machine. |
| Version Control & Docs | Git + GitHub + Mermaid | Mermaid diagrams in README document CP/AP transaction flows; GitHub Actions CI asserts DI=0 in CP mode automatically. |

# 6. Technology Deep Dive

## 6.1 Programming Language — Go

**Chosen: Go**

- goroutines are true OS-scheduled, so concurrent replica sessions interleave naturally without GIL limitations.
- net/http stdlib handles REST control-plane; google.golang.org/grpc handles replication RPC — both are first-class Go.
- sync.RWLock and sync/atomic give fine-grained per-shard locking needed for AP/CP quorum switching at the shard level.
- Single static binary: the same Go binary runs on every replica container with zero runtime dependencies.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Python** | GIL prevents true parallel replica threads; PRAGMA-based isolation less direct than Go's explicit sync primitives. |
| **Java** | JVM startup overhead; ExecutorService works but goroutines are lighter and more idiomatic for this use case. |

| | |
|---|---|
| **Rust** | Excellent concurrency but steep async runtime complexity adds overhead for a simulation/demo project. |
| **C++** | No standard DB abstraction layer; manual memory management adds risk in a concurrent replication simulation. |

## 6.2 Communication / RPC — gRPC

**Chosen: gRPC**

- Server-streaming RPC lets the Control Plane push quorum-change events to all replicas simultaneously.
- Bidirectional streaming supports heartbeat + health check over a single persistent connection per replica pair.
- Protobuf-typed service definitions mean quorum toggle commands are schema-validated — no stringly-typed REST errors.
- gRPC interceptors add request-level tracing (trace_id per write) with zero application code changes.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **HTTP/REST + JSON** | Polling for quorum changes adds latency; no streaming; JSON parsing overhead on the hot replication path. |
| **Thrift** | Facebook RPC — heavier toolchain, poorer Go ecosystem, no streaming support out of the box. |
| **Aeron** | Ultra-low-latency transport for financial systems; our bottleneck is RocksDB I/O, not transport latency. |

## 6.3 Message Bus — NATS JetStream

**Chosen: NATS (JetStream)**

- JetStream subjects like kv.replicate carry put/delete events from primary to replicas with at-least-once delivery.
- NATS runs as a single lightweight Docker container — no ZooKeeper, no broker cluster, no authentication setup needed.
- JetStream persists the event stream so a partitioned replica can replay all missed writes on reconnect in HLC order.
- nats.go is the reference Go client — first-class API, same language as the KV node code, zero bridging needed.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Apache Kafka** | Designed for millions of events/sec; ZooKeeper/KRaft setup overhead is unjustified for a 3-node demo. |
| **RabbitMQ** | AMQP protocol overhead; no native stream replay — a partitioned replica loses events without a custom dead-letter handler. |
| **Redis Streams** | Redis is used for metrics event bus in many stacks; NATS keeps concerns cleaner and avoids dual Redis roles. |

## 6.4 Storage Engine — RocksDB

**Chosen: RocksDB**

- LSM-tree architecture gives high write throughput — critical when AP mode batches writes without quorum delay.
- Native WAL (Write-Ahead Log) records every put/delete before applying; the WAL is the before/after state source for slog.
- Column families allow one RocksDB instance to hold both the KV data and the replication event log in isolated namespaces.
- gorocksdb Go bindings give idiomatic access; Snapshot API enables point-in-time read consistency checks by the Scorer.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **SQLite** | Single-writer WAL mode blocks concurrent replica writes; not designed for multi-session KV replication. |
| **LMDB** | Single-writer only — cannot demonstrate AP mode where multiple writers proceed simultaneously. |
| **LevelDB** | No column families, no transactions; RocksDB is the maintained successor with richer API for this use case. |

## 6.5 Distributed Coordination — etcd

**Chosen: etcd**

- etcd is the definitive CP coordination store — intentionally ironic: AdaptaKV's own mode toggle is strongly consistent.
- etcd watches let every KV node react to /adaptakv/mode key changes within milliseconds — no polling needed.
- etcd leases detect node failures; a failed replica's lease expiry triggers automatic quorum recalculation on surviving nodes.

- etcdctl is a battle-tested CLI for inspecting and manually overriding quorum state during live demos.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **ZooKeeper** | Java-based; JVM dependency; etcd's HTTP API and Go client are simpler and lighter to integrate. |
| **Consul** | Service mesh features (mesh, intentions) are unused; etcd is leaner and purpose-built for key-value coordination. |

## 6.6 Logging / WAL — slog (Go stdlib)

**Chosen: slog (Go standard library)**

- slog is Go's standard structured logger — zero external dependency, zero Log4Shell-class CVE risk.
- JSON handler emits {time, level, node_id, key, before_value, after_value, hlc, quorum_mode} automatically on every write.
- slog.With() attaches per-request trace_id so every log line is correlated to a specific client operation.
- Log files serve as the WAL equivalent — the Correctness Scorer reads log events to compute the Divergence Index.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Custom + Log4j** | Log4j (Java) has critical CVE history (Log4Shell); slog is the safe, zero-dependency Go choice. |
| **structlog (Python)** | Python-only; requires a separate runtime alongside Go — unnecessary complexity. |
| **Apache Log4j** | Java ecosystem only; incompatible with Go runtime without an IPC bridge. |

## 6.7 Serialization — Protobuf

**Chosen: Protobuf**

- KVRequest{key, value, hlc_timestamp, quorum_mode} defined in .proto — auto-generated Go structs, no hand-parsing.
- Binary encoding is ~5x smaller than JSON; critical when replication traffic peaks during AP burst writes.
- Schema evolution: adding hlc_epoch or vector_clock fields in a new .proto does not break existing replicas.

- protoc-gen-go generates both the data types and gRPC service stubs from one .proto file — single source of truth.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **JSON** | Verbose; 'before_value'/'after_value' strings add overhead on the hot replication path; no schema enforcement. |
| **FlatBuffers** | Zero-copy access is beneficial for game engines, not KV replication; more complex Go codegen with marginal benefit. |
| **MessagePack** | Compact binary but no schema definition; no auto-generated Go types; no schema versioning support. |

## 6.8 Concurrency Control — Custom in-memory + Go sync.RWLock

**Chosen: Custom shard map with sync.RWLock per shard**

- 256-shard hash map with one RWLock per shard: readers in AP mode never block each other across different key shards.
- In CP mode, the write path acquires the shard write-lock, replicates to quorum, then releases — serialized per shard.
- sync/atomic CAS on version counters enables optimistic concurrency for read-your-writes guarantees within a session.
- No external lock manager dependency — the entire lock state is in-process, observable via Go's /debug/vars endpoint.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Mutex only (no RW)** | Single mutex per store serializes reads — unacceptable in AP mode where read throughput is the primary goal. |
| **Distributed lock (SETNX)** | Cross-node locking adds network RTT to every write — defeats the AP latency goal entirely. |

## 6.9 Time / Timestamp Oracle — Hybrid Logical Clock (HLC)

**Chosen: Hybrid Logical Clock (HLC)**

- HLC = max(wall_clock, last_seen_HLC) + counter: monotonically increasing even across NTP clock jumps or adjustments.
- Every KV write is stamped with its HLC value; the Correctness Scorer uses HLC comparison to detect stale reads across replicas.
- HLC is gossip-propagated in replication messages via NATS — no central timestamp server, no single point of failure.

- HLC divergence between primary and replica is a direct proxy for replication lag — graphed live in Grafana as hlc_lag_ms.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Google TrueTime** | Requires atomic clock hardware (Google datacenters); not available in a Docker Compose environment. |
| **Local monotonic clock** | Per-node only; provides no cross-node ordering — cannot detect stale reads across replicas. |

## 6.10 Testing / Simulation — Toxiproxy

**Chosen: Toxiproxy**

- Toxiproxy sits as a TCP proxy between KV node containers; adds precise latency (e.g. 300ms on Node A→Node B) to create partition windows.
- REST API controlled from Go test code — no manual iptables rules, no code changes to the KV nodes themselves.
- Deterministic: the same Toxiproxy config produces the same partition scenario every demo run — reproducible for grading.
- Supports bandwidth limits and connection resets — models both slow network (AP scenario) and full partition (CP block scenario).

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Jepsen** | Full distributed systems verifier (Clojure); massive overkill for a 3-node Docker Compose simulation. |
| **Chaos Mesh** | Kubernetes-native chaos engineering tool; our simulation runs on Docker Compose, not Kubernetes. |

## 6.11 Metrics / Dashboard — Prometheus + Grafana

**Chosen: Prometheus + Grafana**

- Go Prometheus client (client_golang) defines counters stale_reads_total{node, mode} and histograms write_latency_seconds{mode}.
- Grafana panel: X-axis = time, Y-axis left = latency_p99, Y-axis right = divergence_index — one view for the full trade-off story.
- Both run as Docker containers — zero cloud accounts, entirely local, self-contained reproducible demo environment.
- Prometheus alerting rule: DI > 0.1 fires an alert — models real SRE alerting on excessive inconsistency in production.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Custom Flask + Plotly** | Requires a separate Python web server alongside Go KV nodes — mixed runtimes add unnecessary complexity. |
| **Datadog / New Relic** | SaaS tools require cloud accounts and network egress; Prometheus + Grafana is purpose-built, free, and local. |

## 6.12 Container / Orchestration — Docker + Docker Compose

**Chosen: Docker + Docker Compose**

- docker-compose.yml defines all 8 services: kv-node-a/b/c, toxiproxy, nats, etcd, prometheus, grafana — one file to rule them all.
- docker compose up reproduces the full 8-container cluster on any machine without manual installs or environment setup.
- Each KV node container has its own network interface — Toxiproxy intercepts traffic exactly as it would in a real distributed system.
- Environment variables (QUORUM_MODE, REPLICA_PEERS) in Compose allow side-by-side CP/AP comparison runs without code changes.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **Kubernetes** | Production-scale orchestration; kubectl + cluster + namespaces = enormous setup overhead for a 3-node demo. |
| **Bare metal / VMs** | Non-reproducible; no isolation between node processes; harder to demonstrate clean partition boundaries. |

## 6.13 Version Control & Docs — Git + GitHub + Mermaid

**Chosen: Git + GitHub + Mermaid diagrams in Markdown**

- Git commit per quorum configuration change: git log shows the exact isolation settings that produced each anomaly run.
- Mermaid sequence diagrams in README document the CP write flow and AP write flow side-by-side — rendered natively in GitHub.
- GitHub Actions CI runs docker compose up + a Go integration test that asserts DI=0 in CP mode — automated correctness gate.
- Markdown ADRs (Architecture Decision Records) document each tech choice with rationale, traceable in git blame.

**Why NOT:**

| Alternative | Reason Rejected |
|---|---|
| **SVN / Mercurial** | Poor branching support; no modern CI/CD integration; the industry has moved entirely to Git. |
| **Confluence + Lucidchart** | Paid tools requiring separate accounts; Mermaid in Markdown achieves the same result for free, inside the repo. |

# 7. How the Chosen Tech Builds AdaptaKV

Each technology plays a specific, non-redundant role in the complete system pipeline:

| Technology Cluster | What It Does in AdaptaKV |
|---|---|
| Go + RocksDB + slog | Each node is one Go binary. RocksDB stores KV pairs with WAL. slog emits {key, before, after, hlc, mode} JSON on every write — this IS the audit trail and WAL. |
| gRPC + Protobuf + etcd | Control Plane calls SetMode(CP\|AP). etcd /adaptakv/mode updated — all node watches fire within ms. Quorum (W, R) recomputed atomically per node. |
| NATS JetStream + HLC | Primary publishes to kv.replicate with HLC timestamp. Replicas consume and apply. On reconnect post-partition, replicas replay missed JetStream events in HLC order. |
| Toxiproxy + Scorer | Toxiproxy injects partition. Scorer polls all 3 nodes for same key. DI = fraction with stale HLC. DI pushed to Prometheus as divergence_index gauge. |
| Prometheus + Grafana | write_latency_seconds{mode='AP'} vs {mode='CP'} — one panel. divergence_index{node} — second panel. Toggle events appear as vertical time annotations. |
| Docker Compose + Git + Mermaid | docker compose up starts all 8 services. README Mermaid diagrams show CP/AP write flows. GitHub CI asserts DI=0 in CP mode on every push. |

# 8. Conclusion

AdaptaKV demonstrates concretely that the CP/AP choice is not a binary deploy-time commitment — it is a live, observable runtime variable. The following outcomes are achieved:

- Demonstrated CP/AP toggle via gRPC SetMode() with near to zero-downtime quorum reconfiguration across 3 RocksDB replicas.
- Logged every read/write as structured JSON via Go slog: {key, before, after, hlc, quorum_mode, node_id}.
- Replicated state using NATS JetStream with HLC-ordered event replay for automatic partition recovery.
- Measured Divergence Index (DI) via Correctness Scorer — a novel, continuous stale-read anomaly metric not in existing production KV stores.
- Learnt about write_latency_p99 vs DI per mode per partition event in a Grafana dashboard.
- Reproduced deterministic partitions using Toxiproxy TCP injection; entire system starts with docker compose up.